Swarm Auditor: The Digital Courtroom

Interim Submission Report

FDE/TRP1 Challenge Week 2: The Automaton Auditor

Birkity Yishak

10 Academy

https://github.com/Birkity/automaton-auditor-fde-week2

February 2026

Abstract

This report documents the architecture, design decisions, and implementation status of **Swarm Auditor**, a hierarchical multi-agent system built with LangGraph that audits GitHub repositories and PDF reports against a 10-dimension rubric. The system follows a "Digital Courtroom" metaphor: forensic Detective agents collect objective evidence using AST parsing and PDF analysis, three adversarial Judge agents (Prosecutor, Defense, Tech Lead) render structured opinions via LLM, and a deterministic Chief Justice synthesizes a final verdict using five hardcoded conflict resolution rules. All three layers are implemented and verified by 165 automated tests. This report explains the reasoning behind each architectural decision and identifies gaps planned for the final submission.

Contents

1 Introduction **2**

2 Architecture Decisions **2**
2.1 Why Pydantic Over Plain Dicts 2
2.2 AST Parsing over Regex 3
2.3 Sandboxing Strategy 3

3 The Rubric as a Machine-Readable Constitution **3**

4 StateGraph Architecture and Flow **3**
4.1 Node Summary 4

5 The Judicial Layer: Dialectical Synthesis **4**

6 The Chief Justice: Deterministic Conflict Resolution **5**

7 Implementation Status **6**

8 Known Gaps and Plan for Final Submission **6**

9 Technology Stack **7**

10 Repository Structure **8**

11 Conclusion **8**

1 Introduction

The FDE/TRP1 Week 2 challenge requires building an *Automated Auditor Swarm*: a system that accepts a GitHub repository URL and a PDF report, runs forensic analysis, evaluates the submission against the 10-dimension "Automaton Auditor" rubric, and produces a structured audit report.

The core insight is that **evaluation is harder than generation**. A single LLM call cannot reliably distinguish between genuinely well-structured code and superficially correct boilerplate. Our solution decomposes the problem into three specialized layers:

1. **Layer 1, The Detective Layer:** Three forensic sub-agents (RepoInvestigator, DocAnalyst, VisionInspector) collect *objective facts* without making judgments. They use deterministic tools (AST parsing, git log extraction, PDF text search) and output structured `Evidence` objects.
2. **Layer 2, The Judicial Layer:** Three LLM-powered judge personas (Prosecutor, Defense, Tech Lead) independently analyze the same evidence for each rubric dimension, producing `JudicialOpinion` objects with scores, arguments, and citations.
3. **Layer 3, The Supreme Court:** A deterministic Chief Justice resolves conflicts between the three judges using five hardcoded Python rules (security override, fact supremacy, functionality weight, variance re-evaluation, dissent requirement), producing a structured `AuditReport`.

This separation is critical: Detectives never opine, Judges never fabricate evidence, and the Chief Justice never relies on LLM reasoning for conflict resolution.

The full source code is available at: `https://github.com/Birkity/automaton-auditor-fde-week2`

2 Architecture Decisions

2.1 Why Pydantic Over Plain Dicts

The challenge specification warns against using "dict soups" for state management. We use Pydantic `BaseModel` classes for every data boundary. This choice has three concrete benefits:

Runtime type safety. When a Detective produces an `Evidence` object, Pydantic validates that `confidence` is a float between 0.0 and 1.0, that `found` is a boolean, and that `dimension_id` is a non-empty string. Without this, a bug returning `confidence="high"` instead of `confidence=0.85` would silently corrupt the pipeline.

LLM structured output binding. The Judicial Layer uses `ChatOllama.with_structured_output(Judi` This method generates a JSON schema from the Pydantic model, injects it into the LLM prompt, and validates the response automatically. With plain dicts, we would need brittle manual parsing.

State reducers for parallel safety. The `AgentState` uses `Annotated` type hints with `operator.ior` (dict merge) and `operator.add` (list append) as LangGraph reducers. When three Detective nodes run in parallel and each returns {`"evidences": {dim_id: [Evidence]}`}, LangGraph merges the dicts without any node overwriting another's results:

Listing 1: AgentState with reducers in `src/state.py`

```python
class AgentState(TypedDict):
    repo_url: str
    pdf_path: str
    rubric_dimensions: List[Dict]
    evidences: Annotated[Dict[str, List[Evidence]], operator.ior]
    opinions: Annotated[List[JudicialOpinion], operator.add]
    final_report: AuditReport
```

Without these reducers, the last node to complete would silently overwrite the results of the other two, discarding two-thirds of the collected evidence.

2.2 AST Parsing over Regex

The challenge requires verifying whether a target repository uses `StateGraph`, has fan-out/fan-in wiring, uses Pydantic models, and implements structured output. A regex search for `"StateGraph"` would produce false positives from comments or string literals. Instead, we use Python's built-in `ast` module to parse source files into Abstract Syntax Trees and walk the tree nodes programmatically across seven forensic protocols:

- **State definitions:** Walk `ast.ClassDef` nodes, extract base class names, flag classes inheriting `BaseModel` or `TypedDict`. Scan `ast.Subscript` nodes for `Annotated` with `operator.add/operator.ior` to detect reducers.
- **Graph structure:** Find `StateGraph()` instantiations, extract all `add_node()` and `add_edge()` calls. Use `collections.Counter` on edge sources/targets to detect fan-out (source count ≥ 2) and fan-in (target count ≥ 2).
- **Tool safety:** Detect `os.system()` calls (security violation), `tempfile.TemporaryDirectory()` usage (sandbox confirmation), and `subprocess.run()` wrapped in `ast.Try` blocks (error handling confirmation).

All seven analysis functions in `repo_tools.py` (1,104 lines) are purely deterministic. They require no LLM and produce identical output for identical input, making them fast, testable, and impossible to hallucinate.

2.3 Sandboxing Strategy

Since the agent clones unknown GitHub repositories, security is a primary concern. We implement three layers of protection:

Temporary directory isolation. Every `clone_repo()` call creates a `tempfile.TemporaryDirectory(pre` The `ClonedRepo` dataclass implements the context manager protocol, ensuring `cleanup()` runs even if an exception occurs during analysis.

Input sanitization. Before passing the URL to `git clone`, we reject any URL containing shell injection characters (`; | & $ ‘`). This prevents attacks like `https://evil.com; rm -rf /` from reaching `subprocess`.

Constrained subprocess execution. We never use `os.system()`. All shell commands run through `subprocess.run()` with `capture_output=True`, `timeout=120`, and manual return code inspection for graceful error handling.

3 The Rubric as a Machine-Readable Constitution

The 10-dimension rubric is stored in `rubric.json` (v3.0.0) and loaded at runtime by the `context_builder` node. Each dimension includes a stable `id`, a `target_artifact` indicating which Detective handles it, a `forensic_instruction` describing what to look for, and `success_pattern/failure_pattern` for evidence interpretation.

Five synthesis rules govern the Chief Justice's conflict resolution: `security_override`, `fact_supremacy`, `functionality_weight`, `dissent_requirement`, and `variance_re_evaluation`. These are hardcoded as deterministic Python logic, not LLM prompts. The scoring rules cannot drift based on prompt phrasing or model temperature; they execute identically every time.

4 StateGraph Architecture and Flow

The full pipeline is a LangGraph `StateGraph` with **9 real nodes** organized in two parallel fan-out/fan-in layers (11 total including the implicit `__start__` and `__end__` sentinels).

Parallelism is achieved through LangGraph's edge-based scheduling. Multiple `add_edge()` calls from one source node tell LangGraph to execute all targets concurrently. The fan-in node

Table 1: Rubric dimensions and their assigned detectives

Dimension	Name	Detective
`git_forensic_analysis`	Git Forensic Analysis	RepoInvestigator
`state_management_rigor`	State Management Rigor	RepoInvestigator
`graph_orchestration`	Graph Orchestration Architecture	RepoInvestigator
`safe_tool_engineering`	Safe Tool Engineering	RepoInvestigator
`structured_output_enforcement`	Structured Output Enforcement	RepoInvestigator
`judicial_nuance`	Judicial Nuance and Dialectics	RepoInvestigator
`chief_justice_synthesis`	Chief Justice Synthesis Engine	RepoInvestigator
`theoretical_depth`	Theoretical Depth (Documentation)	DocAnalyst
`report_accuracy`	Report Accuracy (Cross-Reference)	DocAnalyst
`swarm_visual`	Architectural Diagram Analysis	VisionInspector

(e.g., `evidence_aggregator`) waits until all upstream nodes complete, and the `operator.ior` reducer on the `evidences` field merges parallel outputs into a single dict. This pattern is used twice: once for the three Detectives, once for the three Judges.

Listing 2: Fan-out/fan-in wiring in `src/graph.py`

```
# Detective fan-out
builder.add_edge("context_builder", "repo_investigator")
builder.add_edge("context_builder", "doc_analyst")
builder.add_edge("context_builder", "vision_inspector")
# Detective fan-in
builder.add_edge("repo_investigator", "evidence_aggregator")
builder.add_edge("doc_analyst", "evidence_aggregator")
builder.add_edge("vision_inspector", "evidence_aggregator")
# Judge fan-out
builder.add_edge("evidence_aggregator", "prosecutor")
builder.add_edge("evidence_aggregator", "defense")
builder.add_edge("evidence_aggregator", "tech_lead")
# Judge fan-in
builder.add_edge("prosecutor", "chief_justice")
builder.add_edge("defense", "chief_justice")
builder.add_edge("tech_lead", "chief_justice")
```

4.1 Node Summary

5 The Judicial Layer: Dialectical Synthesis

A single LLM evaluating code tends to produce bland scores in the 3 to 4 range regardless of quality. The dialectical approach forces the model into three distinct reasoning frames:

The **Prosecutor** assumes the code is guilty until proven otherwise: "Start from the assumption that the submission is lazy and has taken shortcuts. Your default score is 1 to 2." The **Defense** acts as an advocate: "Look for creative workarounds, deep understanding, and effort. Your default is 4 to 5 if basic structure is present." The **Tech Lead** provides the pragmatic middle ground: "Ignore the vibe. Does the code actually work? Start at 3 and adjust on concrete evidence."

The three prompt templates share less than 50% word overlap (verified by automated test) to prevent "Persona Collusion," where differently named judges produce near-identical outputs.

Every Judge LLM call uses `ChatOllama.with_structured_output(JudicialOpinion)`, constraining the LLM to return JSON matching the Pydantic schema (judge name, criterion ID, score 1 to 5, argument, cited evidence). If the LLM returns malformed output, a retry loop (up to 2 retries) appends a correction message. If all retries fail, the system falls back to a neutral

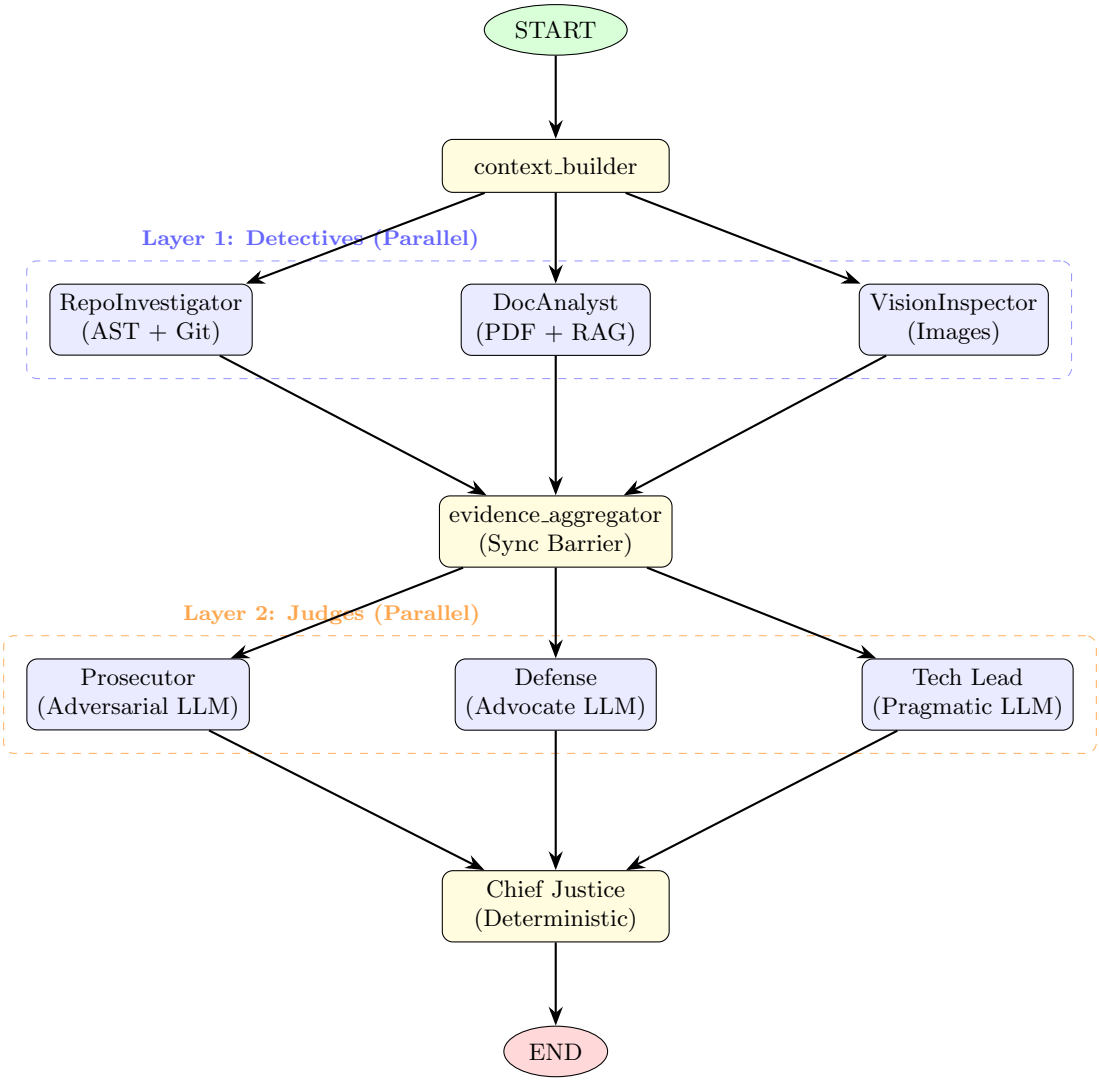

Figure 1: StateGraph topology. Layer 1 uses deterministic Python tools; Layer 2 uses LLM structured output. The Chief Justice applies deterministic conflict resolution.

score of 3 with an error explanation, ensuring the pipeline always completes.

Listing 3: JudicialOpinion schema enforced on every LLM call

```
class JudicialOpinion(BaseModel):
    judge: Literal["Prosecutor", "Defense", "TechLead"]
    criterion_id: str
    score: int = Field(ge=1, le=5)
    argument: str
    cited_evidence: List[str]
```

6 The Chief Justice: Deterministic Conflict Resolution

The Chief Justice is **not an LLM**. It is pure Python `if/elif/else` logic implementing five named rules from the rubric's `synthesis_rules` specification. For each dimension, the base score is a weighted average:

$$\text{weighted} = w_P \times s_P + w_D \times s_D + w_{TL} \times s_{TL}$$

where default weights are w_P=0.30, w_D=0.30, w_{TL}=0.40. The five rules then apply in sequence:

Table 2: Responsibilities of each graph node

Node	Brain	Responsibility
context_builder	Python	Loads rubric dimensions from `rubric.json`, injects into state
repo_investigator	Python/AST	Sandboxed git clone, 7 forensic protocols (git history, state defs, graph structure, tool safety, structured output, judicial nuance, chief justice synthesis)
doc_analyst	PyMuPDF	PDF ingestion with RAG-lite chunking (1000 char), keyword depth analysis, cross-referencing file paths against repo to detect hallucinated claims
vision_inspector	Python	Image extraction from PDF, diagram classification (currently stub)
evidence_aggregator	Python	Sync barrier; logs stats, does not modify state (merging via reducer)
prosecutor	LLM	Adversarial persona; default score 1 to 2; structured output enforced
defense	LLM	Advocate persona; default score 4 to 5 if structure present
tech_lead	LLM	Pragmatic persona; starts at 3, adjusts on technical findings
chief_justice	Python	Deterministic synthesis: 5 conflict resolution rules → `AuditReport`

Rule 1: Security Override. If the RepoInvestigator found `os.system()` or `shell=True` (a concrete AST finding) and the dimension is security-related, the score is capped at 3 regardless of Defense arguments.

Rule 2: Fact Supremacy. If the Defense scored ≥ 4 but every piece of Detective evidence for that dimension has `found=False`, the Defense is overruled for "hallucination" and excluded from the weighted average (Prosecutor 0.40, Tech Lead 0.60).

Rule 3: Functionality Weight. For the `graph_orchestration` dimension, the Tech Lead's weight increases to 0.60, while Prosecutor and Defense drop to 0.20 each. This implements the rubric directive that the Tech Lead carries the highest weight for architecture.

Rule 4: Variance Re-evaluation. When the spread between the highest and lowest judge scores exceeds 2, the final score is pulled toward the median: final $= \lfloor(\text{weighted} + \text{median})/2\rfloor$, dampening extreme disagreements.

Rule 5: Dissent Requirement. When score variance exceeds 2, the system generates a `dissent_summary` documenting each judge's score and argument, included in the final report for transparency. The rubric mandates this: "every criterion with a score variance > 2 must include an explicit dissent explanation."

The final score per dimension is clamped to $[1, 5]$, and the overall report includes an executive summary, per-criterion breakdown with dissent, and a prioritized remediation plan.

7 Implementation Status

8 Known Gaps and Plan for Final Submission

VisionInspector is a stub. It extracts images from PDFs via PyMuPDF but uses placeholder classification logic. For the final submission, we will integrate a multimodal LLM (e.g., `llama3.2-vision`) to classify diagrams as "accurate StateGraph," "sequence diagram," or "misleading linear pipeline."

Self-audit and peer audit not yet generated. The `audit/` directories exist but are

Table 3: File-level implementation status (165 tests passing)

Path	Status	Lines	Description
`src/state.py`	Complete	216	5 Pydantic models + AgentState TypedDict
`src/graph.py`	Complete	121	9-node StateGraph, 2 parallel layers
`src/prompts.py`	Complete	162	3 persona prompt templates + registry
`src/tools/repo_tools.py`	Complete	1104	7 AST forensic protocols, sandboxed git
`src/tools/doc_tools.py`	Complete	440	PDF ingestion, keyword search, cross-ref
`src/tools/vision_tools.py`	Stub	88	Image extraction, low-confidence output
`src/nodes/detectives.py`	Complete	311	3 detective LangGraph nodes
`src/nodes/judges.py`	Complete	200	3 LLM judge nodes with retry logic
`src/nodes/chief_justice.py`	Complete	375	5 synthesis rules + report builder
`app.py`	Complete	460	Streamlit UI with 3 tabs
`rubric.json`	Complete		10 dimensions, 5 synthesis rules
`pyproject.toml`	Complete		uv-managed locked dependencies
`.env.example`	Complete		All env vars documented
Total source		**~3,500**	

empty. We will run the full pipeline against our own repository and our assigned peer's repository, then commit the generated Markdown reports.

Conditional error edges. The graph currently handles errors inside each node via try/except, but has no graph-level conditional edges for short-circuiting (e.g., skip to aggregator if clone fails). Planned for final.

LangSmith tracing. The `.env.example` includes `LANGCHAIN_TRACING_V2=true`, but we have not yet captured a traced end-to-end execution. We will provide a LangSmith trace link in the final submission.

Dockerization. No `Dockerfile` exists yet. We plan to containerize the application for reproducible execution.

Git history improvement. The commit history currently has only 3 commits. We will commit more incrementally going forward to demonstrate iterative development.

9 Technology Stack

Table 4: Core dependencies

Package	Version	Role
`langgraph`	1.0.9	StateGraph, parallel node execution, edge-based orchestration
`langchain-core`	1.2.15	Message types (SystemMessage, HumanMessage)
`langchain-ollama`	1.0.1	ChatOllama with `.with_structured_output()`
`pydantic`	2.12.5	Typed models, field validation, JSON schema generation
`pymupdf`	1.27.1	PDF text and image extraction
`streamlit`	1.54.0	Web UI for interactive auditing
`python-dotenv`	1.2.1	Environment variable management
`pytest`	9.0.2	Test framework (165 tests passing)
`uv`	0.10.0	Dependency management with lockfile

LLM inference runs locally via **Ollama**, requiring no cloud API keys. Available models include `qwen3-coder:480b-cloud` and `deepseek-v3.1:671b-cloud`, configurable through the `OLLAMA_MODEL` environment variable.

10 Repository Structure

Listing 4: Submitted repository layout

```
swarm-auditor/
|-- app.py                          # Streamlit frontend
|-- rubric.json                     # Machine-readable rubric (v3.0.0)
|-- pyproject.toml                  # uv-managed dependencies
|-- uv.lock                         # Locked dependency versions
|-- .env.example                    # Environment variable template
|-- README.md                       # Setup and run instructions
|-- src/
|   |-- state.py                    # Pydantic models + AgentState
|   |-- graph.py                    # 9-node StateGraph
|   |-- prompts.py                  # 3 judge persona prompts
|   |-- tools/
|   |   |-- repo_tools.py          # Sandboxed git + AST analysis
|   |   |-- doc_tools.py           # PDF ingestion + cross-reference
|   |   |-- vision_tools.py        # Image analysis (stub)
|   |-- nodes/
|       |-- detectives.py          # 3 detective LangGraph nodes
|       |-- judges.py              # 3 judge LangGraph nodes
|       |-- chief_justice.py       # Deterministic synthesis engine
|-- tests/                          # 8 test files, 165 tests
|-- audit/                          # Empty; populated at final
|-- reports/
    |-- interim_report.pdf          # This document
```

11 Conclusion

Swarm Auditor implements the complete "Digital Courtroom" architecture required by the FDE/TRP1 Week 2 challenge. The Detective Layer provides objective, AST-based forensic evidence across seven protocols. The Judicial Layer generates structured, adversarial opinions through three distinct LLM personas with verified prompt divergence. The Chief Justice resolves conflicts using deterministic Python rules that faithfully implement the rubric's synthesis specifications.

The system is backed by 165 passing tests, type-safe Pydantic models throughout, sandboxed execution, and a clean separation between facts (Detectives), opinions (Judges), and decisions (Chief Justice). The remaining work for the final submission includes VisionInspector integration, self and peer audit generation, conditional error edges, LangSmith tracing, and Dockerization.